Experiment # 9 & 10

Understanding and Implementation of Operator overloading in C++

Objective

Objective of this lab session is to understand operator overloading, why we use overloading and how we overload operators in C++.

Theory

Operators overloading

C++ has ability to provide the operators with a special meaning for a data. The mechanism of giving such special meanings to an operator is known as operator overloading. We can overload the entire C++ operator except following.

1. Class member access operator (.)
2. Scope resolution operator (::)
3. Size operator (sizeof)
4. Conditional operator (? :)

Why do we need operator overloading?

Before we proceed further, you should know the reason that why we do operator overloading. Do we really need it? For instance, compare the following syntaxes to perform addition of two objects a and b of a user defined class fraction (assume class fraction has a member function called add() which adds two fraction objects):

C = a.add(b);

C = a+b;

Which one is easier to use?

Obviously the second one is easy to use and is more meaningful. After overloading the appropriate operators, *you can use objects in expressions in just the same way that you use C++'s built-in data types.*

If you use the operator without providing the mechanism of how these operators are going to perform with the objects of our class, you will get error message

Defining operator overloading

Operator overloading is done with the help of a special function, called **operator function**, which defines the operations that the overloaded operator will perform relative to the class upon which it will work. An operator function is created using the keyword *operator*. Operator functions can be either members or nonmembers of a class. The way operator functions are written differs between member and nonmember functions.

The general format of member operator function is:

```
class_name class_name :: operator op(arglist)
{
function_body // task defined
}
```

Operator Overloading can be done by using **three approaches**, they are

1. Overloading unary operator.
2. Overloading binary operator.
3. Overloading binary operator using a friend function.

Binary Operators

Here is a simple example of operator overloading. This program creates a class called point, which stores x and y coordinate values. The program overloads the + and ++operators relative to this class.

Example #1

```
#include<iostream>
using namespace std;
class overload_dmas{
      private:
            int a;
      public:
            overload_dmas()
            {
                  a=0;
            }
            overload_dmas(int c)
            {
                  a=c;
            }
            //+
            overload_dmas operator +(overload_dmas& d1)
            {
                  overload_dmas d2;
                  d2.a=a+d1.a;
            return d2;
            }

            //-
            overload_dmas operator -(const overload_dmas& d1)
            {
                  overload_dmas d2;
                  d2.a=a-d1.a;
            return d2;
            }

            void display()
            {
            cout<<"\nvalue of a:"<<a<<endl;
            }
};

int main()
{
      overload_dmas d3(30),d4(5);
```

```
        d3.display();
        d4.display();
        overload_dmas d5,d6,d7,d8;
        cout<<"\nadd:";
        d5=d3+d4;
        d5.display();
        cout<<"\nsubtract:";
        d6=d3-d4;
        d6.display();
        return 0;
}
```

Creating Prefix and Postfix Forms of the Increment and decrement Operators/ Unary Operators

Standard C++ allows you to explicitly create separate prefix and postfix versions of the increment or decrement operators. To accomplish this, you must define two versions of the operator++( ) function. One is defined as shown in the foregoing program. The other is declared like this:

point operator++(int x);

If the ++ precedes its operand, the operator++( ) function is called. If the ++ follows its operand, the operator++(int x) is called and x has the value zero.

Example#2

```
#include<iostream>
using namespace std;
class point
{
int x_coordinate, y_coordinate;
public:
point()//default constructor
 {
        x_coordinate=0;
        y_coordinate=0;
}
point(int x, int y) //parameterized constructor
{
x_coordinate = x;
y_coordinate = y;
}
void show() //function
{
cout<<x_coordinate<< " , ";
cout<<y_coordinate<< "\n";
}
void operator++();
};
// Overload prefix ++ for point.
void point::operator++()  //operator++(int) for postfix
{
```

```
x_coordinate++;
y_coordinate++;
}
int main()
{
point ob1(10, 20), ob2( 5, 30);
ob1.show(); // displays 10 20
++ob1;
//ob2++;
ob1.show();
return 0;
}
```

Lab Tasks

1. Create and Implement a C++ program to make a class box. Set values of its data members i.e. length, width and breadth using constructor. Make member function to display the volume of box. Use operator overloading to add, subtract, multiply and divide the data members (using objects in main) length, breadth and width. Show volume of each overloaded operator's object. Write a driver program to test your class.
2. Create and Implement a C++ program to overload >, <, == operators for the class Complex numbers. Take two objects of class and compare both objects. Write a driver program to test your class.
3. Create and Implement a C++ program to overload ++ operators for the class Time.
    - Data members should be hour, minute and seconds.
    - Set data members as zero in default constructor.
    - Take one object in main to set 23:24:59 hrs.
    - Display time in the given format 23:24:59 hrs.
    - Overload ++ operator to make a stop watch. (i.e. that will increment seconds by 1 each time its called)
    - Make a function by name add_time() to increment seconds.
    - Write a driver program to test your class.

Conclusion

_____
_____
_____
_____
_____
_____

_____

Lab Instructor
Kaynat Rana